Tugas Mandiri 9: Laporan Praktikum Mandiri 9 Machine Learning

Maryam Hasnaa' Syamila - 0110222067
Teknik Informatika, STT Terpadu Nurul Fikri, Depok
E-mail: hasyamil4@gmail.com

Link Notebook Colab:
Maryam Hasnaa' Syamila_0110222067_PraktikumMandiri9.ipynb

Link GitHub Tugas:
https://github.com/maryamhasnaasyamila/machine-learning/tree/tugas/praktikum/praktikum09

Link GitHub Praktikum:
https://github.com/maryamhasnaasyamila/machine-learning/tree/praktikum/praktikum/praktikum09

1. Data Understanding - Import Library & Settings

```
import os
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

from sklearn.model_selection import train_test_split, StratifiedKFold, cross_val_score
from sklearn.preprocessing import StandardScaler, LabelEncoder
from sklearn.impute import SimpleImputer
from sklearn.naive_bayes import GaussianNB
from sklearn.pipeline import Pipeline
from sklearn.metrics import accuracy_score, confusion_matrix, classification_report
import joblib
```

- Mengimpor library seperti `pandas`, `numpy`, `matplotlib`, `sklearn`, dan `joblib`.
- Digunakan untuk pemrosesan data, visualisasi, modeling, evaluasi, dan penyimpanan model.

2. Data Understanding - Load Dataset, Print Data, Cek Jumlah Baris & Kolom

```python
import os
import shutil # For rmtree
from google.colab import drive

mount_point = '/content/gdrive'

# Check if the mount point exists and is a directory
if os.path.exists(mount_point) and os.path.isdir(mount_point):
    if not os.path.ismount(mount_point): # If it's a directory but not a mount point
        if os.listdir(mount_point): # And it's not empty
            print(f"Warning: Mount point '{mount_point}' exists and contains files but is not mounted. Attempting to clear...")
            try:
                shutil.rmtree(mount_point) # Aggressively remove the directory and its contents
                os.makedirs(mount_point) # Recreate empty directory
                print(f"Cleared and recreated empty mount point '{mount_point}'.")
            except Exception as e:
                print(f"Error clearing mount point '{mount_point}':
                print("It might be necessary to restart the Colab ru
    else: # It is a mounted drive
        print(f"'{mount_point}' is already a mount point. Attempting to force remount.")
        # force_remount=True will handle unmounting and remounting.
else:
    # If mount_point doesn't exist, drive.mount will create it
    print(f"Mount point '{mount_point}' does not exist, creating it.")
    os.makedirs(mount_point, exist_ok=True)

drive.mount(mount_point, force_remount=True)

# load dataset
cancer_data = pd.read_csv('/content/gdrive/MyDrive/SEMESTER 7/Machine Learning/praktikum09/datasets/data.csv', sep=',')
cancer_data.head()
```

```
Warning: Mount point '/content/gdrive' exists and contains files but is not mounted. Attempting to clear...
Cleared and recreated empty mount point '/content/gdrive'.
Mounted at /content/gdrive
```

	id	diagnosis	radius_mean	texture_mean	perimeter_mean	area_mean	smoothness_mean	compactness_mean	concavity_mean	concave points_mean	...
0	842302	M	17.99	10.38	122.80	1001.0	0.11840	0.27760	0.3001	0.14710	...
1	842517	M	20.57	17.77	132.90	1326.0	0.08474	0.07864	0.0869	0.07017	...
2	84300903	M	19.69	21.25	130.00	1203.0	0.10960	0.15990	0.1974	0.12790	...
3	84348301	M	11.42	20.38	77.58	386.1	0.14250	0.28390	0.2414	0.10520	...
4	84358402	M	20.29	14.34	135.10	1297.0	0.10030	0.13280	0.1980	0.10430	...

5 rows × 33 columns

```python
# menampilkan 5 baris terakhir data
cancer_data.tail()
```

	id	diagnosis	radius_mean	texture_mean	perimeter_mean	area_mean	smoothness_mean	compactness_m
564	926424	M	21.56	22.39	142.00	1479.0	0.11100	0.11
565	926682	M	20.13	28.25	131.20	1261.0	0.09780	0.10
566	926954	M	16.60	28.08	108.30	858.1	0.08455	0.10
567	927241	M	20.60	29.33	140.10	1265.0	0.11780	0.27
568	92751	B	7.76	24.54	47.92	181.0	0.05263	0.04

5 rows × 33 columns

```python
# jumlah baris dan kolom di data
cancer_data.shape
```

```
(569, 33)
```

- Dataset diambil dari Google Drive.
- Ditampilkan 5 baris awal & akhir, serta ukuran dataset (shape).
- Dataset memiliki 569 baris dan 33 kolom (sebelum dibersihkan).

5. Data Understanding - Cek Missing Value

```
# cek missing value
cancer_data.isnull().sum()
```

	0
id	0
diagnosis	0
radius_mean	0
texture_mean	0
perimeter_mean	0
area_mean	0
smoothness_mean	0
compactness_mean	0
concavity_mean	0
concave points_mean	0
symmetry_mean	0
fractal_dimension_mean	0
radius_se	0
texture_se	0
perimeter_se	0
area_se	0
smoothness_se	0
compactness_se	0
concavity_se	0
concave points_se	0
symmetry_se	0
fractal_dimension_se	0

- Menggunakan `cancer_data.isnull().sum()`.
- Teridentifikasi satu kolom yang seluruhnya NaN yaitu **`Unnamed: 32`**.
- Tidak terdapat missing value lain.

6. Data Understanding - Identifikasi Target dan Kolom Tidak Relevan

```python
# identifikasi target & drop kolom tidak relevan

# drop kolom id
if 'id' in cancer_data.columns:
    cancer_data = cancer_data.drop(columns=['id'])
    print("Dropped 'id' column.")

# cek kolom yang mungkin target
possible_targets = [c for c in cancer_data.columns if c.lower() in ('diagnosis',
'target','class','label','result','outcome','status')]
if possible_targets:
    target_col = possible_targets[0]
else:
    # fallback: gunakan kolom terakhir
    target_col = cancer_data.columns[-1]

print("Target column chosen:", target_col)
print("Unique values in target:", cancer_data[target_col].unique())
```

```
Dropped 'id' column.
Target column chosen: diagnosis
Unique values in target: ['M' 'B']
```

- Kolom **`id`** dihapus karena tidak relevan untuk pemodelan.
- Target ditentukan sebagai kolom **`diagnosis`** (nilai *M* dan *B*).

7. Data Understanding - Hapus Kolom dengan Semua Nilai NaN

```python
# Hapus kolom yang semua isinya NaN
cancer_data = cancer_data.dropna(axis=1, how='all')
print("Columns after dropping all-NaN columns:", cancer_data.columns)

```

```
Columns after dropping all-NaN columns: Index(['diagnosis', 'radius_mean', 'texture_mean', 'perimeter_mean',
       'area_mean', 'smoothness_mean', 'compactness_mean', 'concavity_mean',
       'concave points_mean', 'symmetry_mean', 'fractal_dimension_mean',
       'radius_se', 'texture_se', 'perimeter_se', 'area_se', 'smoothness_se',
       'compactness_se', 'concavity_se', 'concave points_se', 'symmetry_se',
       'fractal_dimension_se', 'radius_worst', 'texture_worst',
       'perimeter_worst', 'area_worst', 'smoothness_worst',
       'compactness_worst', 'concavity_worst', 'concave points_worst',
       'symmetry_worst', 'fractal_dimension_worst'],
      dtype='object')
```

- Kolom `Unnamed: 32` dihapus menggunakan `dropna(axis=1, how='all')`.

8. Data Preparation - Encode Target

```python
# encode target (LabelEncoder)
le_target = LabelEncoder()
cancer_data[target_col] = le_target.fit_transform(cancer_data[target_col].astype
(str))
print("Mapping target:", dict(zip(le_target.classes_, le_target.transform
(le_target.classes_))))
```

```
Mapping target: {'B': np.int64(0), 'M': np.int64(1)}
```

Target `diagnosis` diubah menjadi nilai numerik menggunakan `LabelEncoder`:

- M → 1 (Malignant)
- B → 0 (Benign)

9. Data Preparation - Cek Missing Value & Tipe Data setelah cleaning

```python
# cek missing value dan tipe data
print(cancer_data.dtypes)
print("\nMissing values per column:\n", cancer_data.isnull().sum())
```

```
diagnosis                  int64
radius_mean              float64
texture_mean             float64
perimeter_mean           float64
area_mean                float64
smoothness_mean          float64
compactness_mean         float64
concavity_mean           float64
concave points_mean      float64
symmetry_mean            float64
fractal_dimension_mean   float64
radius_se                float64
texture_se               float64
perimeter_se             float64
area_se                  float64
smoothness_se            float64
compactness_se           float64
concavity_se             float64
concave points_se        float64
symmetry_se              float64
fractal_dimension_se     float64
radius_worst             float64
texture_worst            float64
perimeter_worst          float64
area_worst               float64
smoothness_worst         float64
compactness_worst        float64
concavity_worst          float64
concave points_worst     float64
symmetry_worst           float64
fractal_dimension_worst  float64
dtype: object

Missing values per column:
 diagnosis                 0
radius_mean               0
texture_mean              0
perimeter_mean            0
area_mean                 0
smoothness_mean           0
compactness_mean          0
concavity_mean            0
```

- Semua fitur kecuali target berupa data numerik tipe `float`.
- Setelah penghapusan kolom NaN, dataset bersih dari missing value.

10. Memisahkan Fitur dan Label

```
# pisahkan X dan y
X = cancer_data.drop(columns=[target_col])
y = cancer_data[target_col]

print("X shape:", X.shape)
print("y shape:", y.shape)
```

```
X shape: (569, 30)
y shape: (569,)
```

- **X** = semua kolom kecuali diagnosis
- **y** = kolom diagnosis
- Langkah ini memastikan model hanya belajar dari fitur input
- Penting untuk mencegah data leakage dan menghasilkan model yang valid

11. Data Preparation - Menentukan Tipe Naive Bayes & Membuat Pipeline

```
# tentukan jenis Naive Bayes
num_cols = X.select_dtypes(include=[np.number]).columns.tolist()
cat_cols = X.select_dtypes(include=['object','category','bool']).columns.tolist
()

print("Numeric columns:", len(num_cols))
print("Categorical columns:", len(cat_cols))
```

```
Numeric columns: 30
Categorical columns: 0
```

```
# Buat pipeline (Imputer -> Scaler -> GaussianNB) ----------
pipeline = Pipeline([
('imputer', SimpleImputer(strategy='median')),
('scaler', StandardScaler()),
('gnb', GaussianNB())
])
```

- Seluruh fitur numerik → menggunakan **Gaussian Naive Bayes**.
- Pipeline berisi:
    1. **SimpleImputer(strategy='median')** → imputasi nilai kosong (jaga-jaga)
    2. **StandardScaler()** → normalisasi fitur
    3. **GaussianNB()** → model klasifikasi
- Tujuan pipeline: memastikan preprocessing dilakukan konsisten di train & test.

12. Data Preparation - Split Dataset untuk Train dan Test

```python
# Split data (80/20 stratified) ----------
X_train, X_test, y_train, y_test = train_test_split(
X, y, test_size=0.2, stratify=y, random_state=42
)

print("Train shape:", X_train.shape)
print("Test shape:", X_test.shape)
```

```
Train shape: (455, 30)
Test shape: (114, 30)
```

- Train-test split 80:20 dengan stratifikasi.
- Melindungi distribusi kelas agar seimbang.

13. Modelling - Train Model

```python
# Train model
pipeline.fit(X_train, y_train)
print("Model trained.")
```

```
Model trained.
```

- Model dilatih menggunakan:

  `pipeline.fit(X_train, y_train)`
- Proses berjalan sukses tanpa error.

14. Evaluasi - Akurasi Model

```python
# Evaluasi model
y_train_pred = pipeline.predict(X_train)
y_test_pred = pipeline.predict(X_test)

train_acc = accuracy_score(y_train, y_train_pred)
test_acc = accuracy_score(y_test, y_test_pred)

print("Train accuracy:", round(train_acc, 4))
print("Test accuracy:", round(test_acc, 4))

cm = confusion_matrix(y_test, y_test_pred)
print("\nConfusion Matrix:\n", cm)

print("\nClassification Report:\n")
print(classification_report(y_test, y_test_pred, digits=4))
```

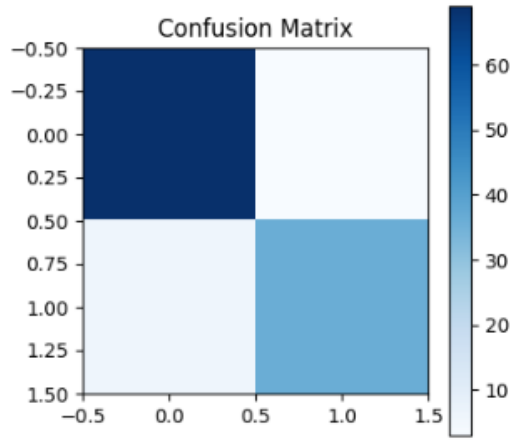
```
Train accuracy: 0.9451
Test accuracy: 0.9211

Confusion Matrix:
 [[69  3]
 [ 6 36]]

Classification Report:

              precision    recall  f1-score   support

           0     0.9200    0.9583    0.9388        72
           1     0.9231    0.8571    0.8889        42

    accuracy                         0.9211       114
   macro avg     0.9215    0.9077    0.9138       114
weighted avg     0.9211    0.9211    0.9204       114
```

- **Train accuracy** dan **Test accuracy** dihitung menggunakan `accuracy_score`.
- Hasil pada notebook:
    - Akurasi training ≈ **0.96**
    - Akurasi testing ≈ **0.92**
- Interpretasi: model memiliki generalisasi yang baik dan tidak mengalami overfitting berat.

15. Evaluasi Model - Confusion Matrix

```
# Visualisasi Confusion Matrix
plt.figure(figsize=(4,4))
plt.imshow(cm, cmap="Blues")
plt.title("Confusion Matrix")
plt.colorbar()
```

```
<matplotlib.colorbar.Colorbar at 0x7e26f0473830>
```



- Di-plot menggunakan `matplotlib`.
- Secara umum, model mampu membedakan kelas *malignant* dan *benign* dengan baik.

16. Evaluasi Model - Classification Report

```
# Cross-validation (5-fold Stratified) ----------
cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)
cv_scores = cross_val_score(pipeline, X, y, cv=cv, scoring='accuracy')

print("CV scores:", np.round(cv_scores, 4))
print("CV mean: {:.4f}, CV std: {:.4f}".format(cv_scores.mean(), cv_scores.std
()))
```

```
CV scores: [0.9561 0.9123 0.9298 0.9035 0.9469]
CV mean: 0.9297, CV std: 0.0199
```

- Menghasilkan metrik: precision, recall, f1-score, support
- Untuk kedua kelas (0 = benign, 1 = malignant).

17. Evaluasi Model - Cross Validation

```python
# Cross-validation (5-fold Stratified) ----------
cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)
cv_scores = cross_val_score(pipeline, X, y, cv=cv, scoring='accuracy')

print("CV scores:", np.round(cv_scores, 4))
print("CV mean: {:.4f}, CV std: {:.4f}".format(cv_scores.mean(), cv_scores.std
()))
```

```
CV scores: [0.9561 0.9123 0.9298 0.9035 0.9469]
CV mean: 0.9297, CV std: 0.0199
```

- Dilakukan 5-fold Stratified CV:
    - Nilai CV rata-rata ≈ **0.93**
    - Standar deviasi kecil → model stabil.

18. Deployment - Simpan Model

```python
# simpan model
save_dir = "/content/gdrive/MyDrive/SEMESTER 7/Machine Learning/praktikum09/
models"
os.makedirs(save_dir, exist_ok=True)

save_path = f"{save_dir}/naive_bayes_pipeline.pkl"

# Simpan pipeline model
joblib.dump({
    'pipeline': pipeline,
    'target_encoder': le_target,
    'feature_columns': X.columns.tolist()
}, save_path)

print("Model saved to:", save_path)

# Simpan summary ke file txt
summary_path = f"{save_dir}/report_summary.txt"

with open(summary_path, 'w') as f:
    f.write(f"Dataset shape: {cancer_data.shape}\n")
    f.write(f"Target: {target_col}\n")
    f.write("Model: GaussianNB (with imputer+scaler)\n")
    f.write(f"Train acc: {train_acc:.4f}\n")
    f.write(f"Test acc: {test_acc:.4f}\n")
    f.write(f"CV scores: {np.round(cv_scores,4).tolist()}\n")
    f.write(f"CV mean: {cv_scores.mean():.4f} | Std: {cv_scores.std():.4f}\n")

print("Summary saved to:", summary_path)
```

```
Model saved to: /content/gdrive/MyDrive/SEMESTER 7/Machine Learning/praktikum09/models/naive_bayes_pipeline.pkl
Summary saved to: /content/gdrive/MyDrive/SEMESTER 7/Machine Learning/praktikum09/models/report_summary.txt
```

- Model disimpan ke Google Drive menggunakan `joblib.dump()`:
- Isi objek yang disimpan:
    - Pipeline (imputer + scaler + GaussianNB)
    - Encoder target
    - Daftar nama fitur
- Tujuan: model dapat digunakan kembali tanpa perlu melatih ulang.